

Labo 18

opleidingsonderdeel **Web Development 1**

studiegebied **Handelswetenschappen, bedrijfskunde**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Sander Vandeputte

docent: **Pim Debaere**

academiejaar **2025-2026**

Opdracht demo events

blablabla 1

blablabla 2

blablabla 3

blablabla 4

blablabla 5

Klik op een paragraaf om diens tekst rood te maken



blablabla 1

blablabla 2

blablabla 3

blablabla 4

blablabla 5

Klik op een paragraaf om diens tekst rood te maken

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
<p class="text">blablabla 1</p>
<p class="text">blablabla 2</p>
<p class="text">blablabla 3</p>
<p class="text">blablabla 4</p>
<p class="text">blablabla 5</p>
Klik op een paragraaf om diens tekst rood te maken
<script src="../scripts/demo_events.js"></script>
</body>
</html>
```

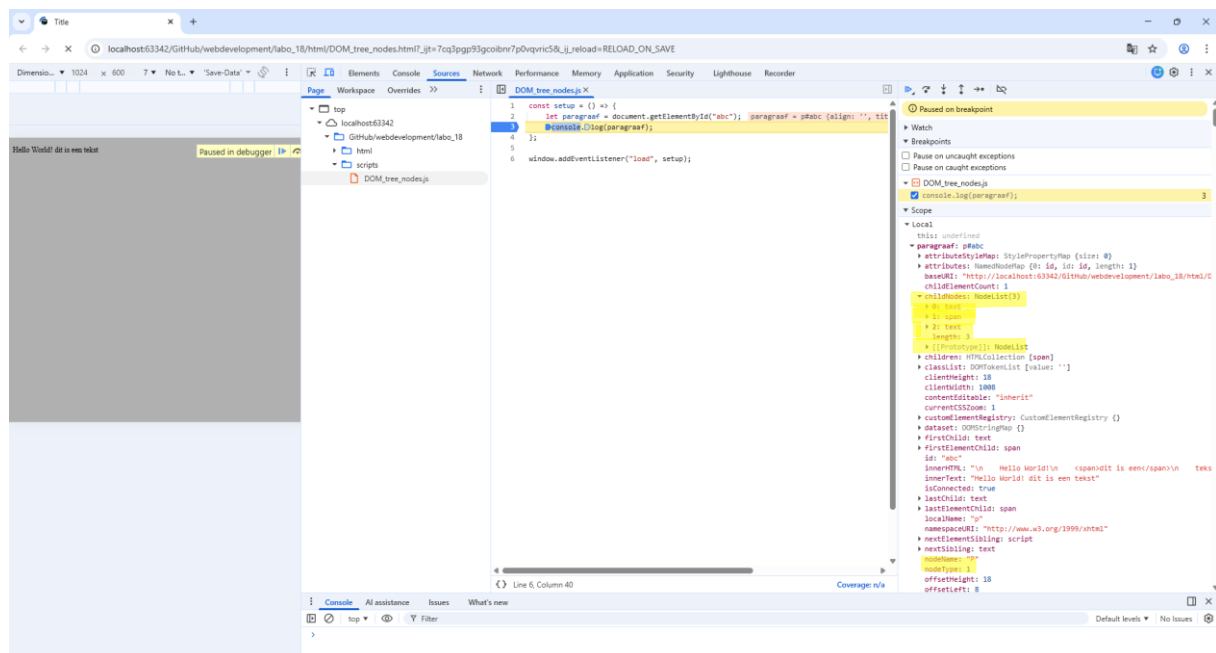
Javascript

```
const setup = () => {
  let texts=document.querySelectorAll(".text");
  for (let i=0;i<texts.length;i++) {
    texts[i].addEventListener("click", klik);
  }
}

const klik = (event) => {
  event.target.style.color="red";
};

window.addEventListener("load", setup);
```

DOM tree nodes



Wat is de precieze inhoud van de TEXT nodes?

Er zijn er 2: "\n Hello World!\n " en "\n tekst\n"

Welke soort nodes kom je tegen in het document?

Hier vind je een **DOCUMENT_NODE** (nodeType 9),
DOCUMENT_TYPE_NODE (nodeType 10),
ELEMENT_NODE (nodeType 1 (bv. html, head, body, p, span, ...))
en **TEKST_NODE** (nodeType 3 (echte tekst + witruimte))

```
0: text
  baseURI: "http://localhost:63342/GitHub/wel
  childNodes: NodeList []
  data: "\n Hello World!\n "
  isConnected: true
  length: 22
  nextElementSibling: span
  nextSibling: span
  nodeName: "#text"
  nodeType: 3
  nodeValue: "\n Hello World!\n "
  ownerDocument: document
  parentElement: p#abc
  parentNode: p#abc
  textContent: "\n Hello World!\n "
  wholeText: "\n Hello World!\n "
  [[Prototype]]: Text
1: span
2: text
  baseURI: "http://localhost:63342/GitHub/wel
  childNodes: NodeList []
  data: "\n tekst\n"
  isConnected: true
  length: 11
  nodeName: "#text"
  nodeType: 3
  nodeValue: "\n tekst\n"
  ownerDocument: document
  parentElement: p#abc
  parentNode: p#abc
  previousElementSibling: span
  previousSibling: span
  textContent: "\n tekst\n"
  wholeText: "\n tekst\n"
  [[Prototype]]: Text
length: 3
[[Prototype]]: NodeList

ownerDocument: document
  location: Location {ancestorOrigins: DOMStringList, href: 'http://localho
  URL: "http://localhost:63342/GitHub/webdevelopment/labo_18/html/DOM_tree_
  activeElement: body
  adoptedStyleSheets: Proxy(Array) {}
  anchors: HTMLCollection []
  applets: HTMLCollection []
  baseURI: "http://localhost:63342/GitHub/webdevelopment/labo_18/html/DOM_tr
  body: body
  characterSet: "UTF-8"
  charset: "UTF-8"
  childElementCount: 1
  childNodes: NodeList(2)
    0: <DOCTYPE html>
    1: html
    length: 2
  [[Prototype]]: NodeList
  children: HTMLCollection [html]
  compatMode: "CSS1Compat"
  contentType: "text/html"
  customElementRegistry: CustomElementRegistry {}
  defaultView: Window {window: Window, self: Window, document: document, nar
  designMode: "off"
  doctype: <DOCTYPE html>
  documentElement: html
  documentURI: "http://localhost:63342/GitHub/webdevelopment/labo_18/html/DX
  domain: "localhost"
  embeds: HTMLCollection []
  featurePolicy: FeaturePolicy {}
  firstChild: <DOCTYPE html>
  firstElementChild: html
  fonts: FontFaceSet {onloading: null, onloadingdone: null, onloadingerror:
  forms: HTMLCollection []
  fragmentDirective: FragmentDirective {}
  fullscreenEnabled: true
  head: head
  images: HTMLCollection []
  implementation: DOMImplementation {}
  inputEncoding: "UTF-8"
  isConnected: true
  lastChild: html
  lastElementChild: html
  lastModified: "03/18/2026 11:16:47"
  links: HTMLCollection []
  nodeName: "#document"
  nodeType: 9
  pictureInPictureEnabled: true
  plugins: HTMLCollection []
  readyState: "complete"
  referrer: "http://localhost:63342/GitHub/webdevelopment/labo_18/html/DOM_1
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>

<p id="abc">
  Hello World!
  <span>dit is een</span>
  tekst
</p>

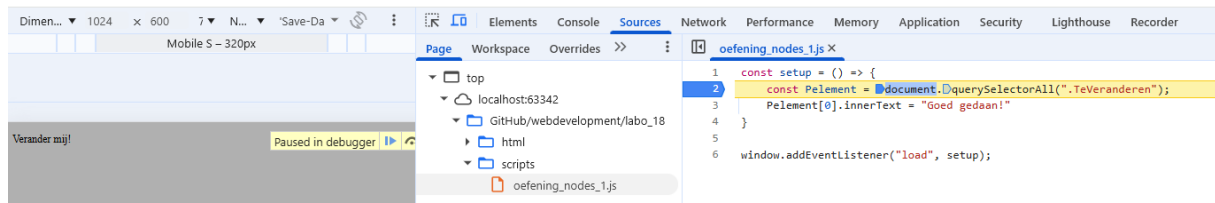
<script src="../../scripts/DOM_tree_nodes.js"></script>
</body>
</html>
```

Javascript

```
const setup = () => {
  let paragraaf = document.getElementById("abc");
  console.log(paragraaf);
};

window.addEventListener("load", setup);
```

oefeningen nodes: oefening 1



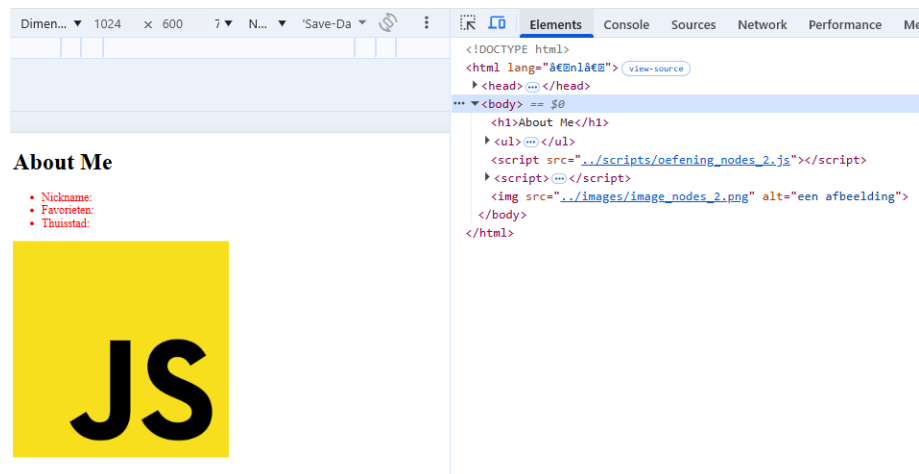
HTML

```
<!DOCTYPE html>  
<html lang="nl">  
<body>  
<p class="TeVeranderen">Verander mij!</p>  
  
<script src="../scripts/oefening_nodes_1.js"></script>  
</body>  
</html>
```

Javascript

```
const setup = () => {  
  const Pelement = document.querySelectorAll(".TeVeranderen");  
  Pelement[0].innerText = "Goed gedaan!"  
}  
  
window.addEventListener("load", setup);
```

oefeningen nodes: oefening 2



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="utf-8"/>
  <title>Over mij</title>
</head>
<body>
<h1>About Me</h1>
<ul>
  <li>Nickname: <span id="nickname"></span>
  <li>Favorieten: <span id="favorites"></span>
  <li>Thuisstad: <span id="hometown"></span>
</ul>

<script src="../scripts/oefening_nodes_2.js"></script>
</body>
</html>
```

Javascript

```
const setup = () => {
  const listItem = document.querySelectorAll('li');
  listItem.forEach(item => {
    item.setAttribute("class", "listitem");
  })

  const style = document.createElement("style");
  style.innerText = ".listitem {color: red;}";
  document.head.appendChild(style);

  const image = document.createElement("img");
  image.setAttribute("src", "../images/image_nodes_2.png");
  image.setAttribute("alt", "een afbeelding");
  document.body.appendChild(image);
}

window.addEventListener('load', setup)
```

oefeningen nodes: oefening 3

Klik op een knop om een p-element aan te maken.

Dit is een div-element

Toegevoegde paragraaf

Toegevoegde paragraaf

Toegevoegde paragraaf

Toegevoegde paragraaf

klik om tekst toe te voegen

HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <style>
    #myDIV {
      border: 1px solid black;
      margin-bottom: 10px;
    }
  </style>
</head>
<body>
<p>
  Klik op een knop om een p-element aan te maken.
</p>
<div id="myDIV">
  Dit is een div-element
</div>
<button id="toevoegen">klik om tekst toe te voegen</button>

<script src="../../scripts/oefening_nodes_3.js"></script>
</body>
</html>
```

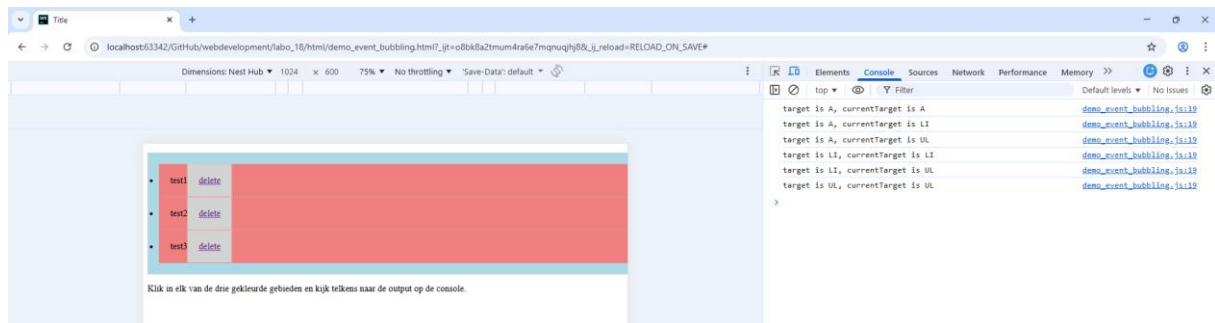
Javascript

```
const setup = () => {
  document.querySelector("button").addEventListener("click", toevoegenP);
}

const toevoegenP = () => {
  const p = document.createElement("p");
  p.textContent = "Toegevoegde paragraaf";
  document.querySelector("#myDIV").appendChild(p);
}

window.addEventListener('load', setup)
```

demo event bubbling



HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <link rel="stylesheet" href="../styles/demo_event_bubbling.css">
</head>
<body>
<ul>
  <li>test1<a href="#">delete</a></li>
  <li>test2<a href="#">delete</a></li>
  <li>test3<a href="#">delete</a></li>
</ul>
Klik in elk van de drie gekleurde gebieden en kijk telkens naar de output
op de console.
<script src="../scripts/demo_event_bubbling.js"></script>
</body>
</html>
```

CSS

```
ul {
  padding:20px;
  background-color:lightblue;
}

li {
  padding:20px;
  background-color:lightcoral;
}

a {
  padding:20px;
  background-color:lightgrey;
}
```

Javascript

```
const setup = () => {
  let uls=document.getElementsByTagName("ul");
  uls[0].addEventListener("click", klik);

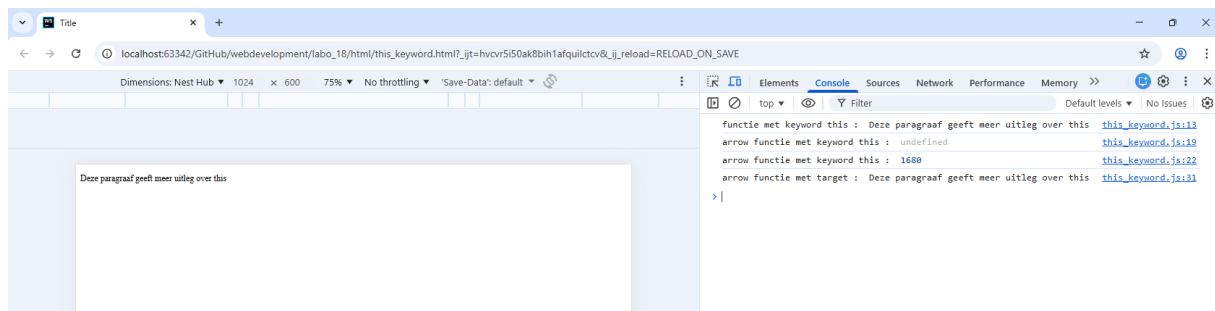
  let lis=document.getElementsByTagName("li");
  for (let i=0;i<lis.length;i++) {
    lis[i].addEventListener("click", klik);
  }

  let as=document.getElementsByTagName("a");
  for (let i=0;i<as.length;i++) {
    as[i].addEventListener("click", klik);
  }
};

const klik = (event) => {
  console.log("target is "+event.target.nodeName+", currentTarget is "+event.currentTarget.nodeName);
};

window.addEventListener("load", setup);
```

Demo this keyword



Javascript

```
const setup = () => {
    document.getElementById("txtParagraaf").addEventListener("click", show);

    document.getElementById("txtParagraaf").addEventListener("click", showArrowFunctie);

    document.getElementById("txtParagraaf").addEventListener("click", showArrowFunctieTarget);
};
window.addEventListener("load", setup);

// functie NIET in een arrow notatie
function show ()
{
    //In event handlers, this refers to the HTML element that received the event:
    console.log("functie met keyword this : ", this.innerHTML);
}

// functie in een arrow notatie
const showArrowFunctie = () =>
{
    console.log("arrow functie met keyword this : ", this.innerHTML); // je zal undefined krijgen -> je krijgt de Window-object terug
    // uitleg: https://wesbos.com/arrow-functions-this-javascript/

    console.log ("arrow functie met keyword this : ", this.innerWidth); // -> je zal de breedte van het scherm krijgen (innerWidth is een property van het Window object)
};

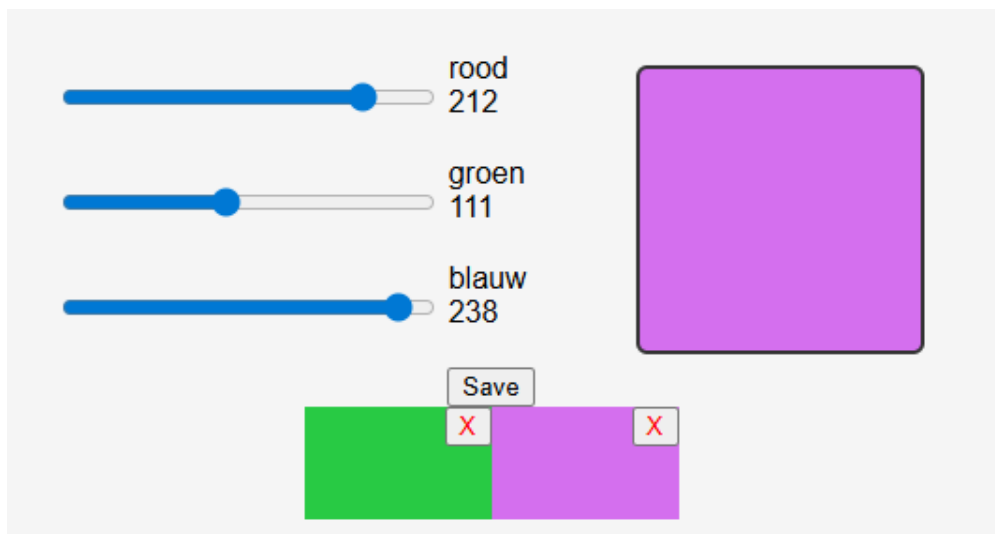
//oplossing
// functie in een arrow notatie
const showArrowFunctieTarget = (event) => {

    console.log("arrow functie met target : ", event.target.innerHTML);
}
;
```

Html

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
  <p id="txtParagraaf">Deze paragraaf geeft meer uitleg over this</p>
  <script src="../scripts/this_keyword.js"></script>
</body>
</html>
```

oefening colorpicker uitbreiding



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Colorpicker</title>
  <link rel="stylesheet" href="../../styles/colorpicker.css">
</head>
<body>

<div class="container">
  <div class="sliders">

    <div>
      <input type="range" class="rood_slider" min="0" max="255">
      <span>
        <span>rood</span>
        <span id="rood_value"> 255 </span>
      </span>
    </div>
    <div>
      <input type="range" class="groen_slider" min="0" max="255">
      <span>
        <span>groen</span>
        <span id="groen_value"> 128 </span>
      </span>
    </div>
    <div>
      <input type="range" class="blauw_slider" min="0" max="255">
      <span>
        <span>blauw</span>
        <span id="blauw_value"> 128 </span></span>
      </span>
    </div>
  </div>
  <div id="color-box"></div>
</div>
<script src="../../scripts/colorpicker.js"></script>
</body>
</html>
```

Javascript

```
const global = {
  huidigeKleur: "rgb(128, 128, 128)"
};

const setup = () => {
  const roodSlider = document.getElementsByClassName("rood_slider")[0];
  const groenSlider = document.getElementsByClassName("groen_slider")[0];
  const blauwSlider = document.getElementsByClassName("blauw_slider")[0];

  const roodValue = document.getElementById("rood_value");
  const groenValue = document.getElementById("groen_value");
  const blauwValue = document.getElementById("blauw_value");

  const colorBox = document.getElementById("color-box");

  const updateColor = () => {
    const r = roodSlider.value;
    const g = groenSlider.value;
    const b = blauwSlider.value;

    roodValue.textContent = r;
    groenValue.textContent = g;
    blauwValue.textContent = b;

    colorBox.style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
    global.huidigeKleur = `rgb(${r}, ${g}, ${b})`;
  };

  const opslaanKnop = document.createElement("button");
  opslaanKnop.textContent = "Save";
  document.querySelector(".container").insertAdjacentElement("afterend",
  opslaanKnop);

  const kleurVlakkenLijst = document.createElement("div");
  document.body.appendChild(kleurVlakkenLijst);

  opslaanKnop.addEventListener("click", () => {
    const r = roodSlider.value;
    const g = groenSlider.value;
    const b = blauwSlider.value;

    const kleurVlak = document.createElement("div");
    kleurVlak.style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
    kleurVlak.style.width = "100px";
    kleurVlak.style.height = "60px";
    kleurVlak.style.display = "inline-block";
    kleurVlak.style.position = "relative";
    kleurVlak.style.cursor = "pointer";

    kleurVlak.addEventListener("click", () => {
      roodSlider.value = r;
      groenSlider.value = g;
      blauwSlider.value = b;
      updateColor();
    });

    const verwijderKnop = document.createElement("button");
    verwijderKnop.textContent = "X";
    verwijderKnop.style.color = "red";
```

```
verwijderKnop.style.position = "absolute";
verwijderKnop.style.top = "0";
verwijderKnop.style.right = "0";
verwijderKnop.addEventListener("click", (e) => {
    e.stopPropagation();
    kleurVlak.remove();
});

kleurVlak.appendChild(verwijderKnop);
kleurVlakkenLijst.appendChild(kleurVlak);
});

roodSlider.addEventListener("input", updateColor);
groenSlider.addEventListener("input", updateColor);
blauwSlider.addEventListener("input", updateColor);

updateColor();
};

window.addEventListener("load", setup);
```